

Final Exam Preparation: Level 3 - 4

Descriptions

Based on the roster detailed in PLAN.pdf, here is the complete breakdown of every possible question for Levels 3 and 4.

Note: Questions are either a **Program** (requires `main`, `argc`, `argv`, and prints to standard output) or a **Function** (you only write the requested function and submit it without a `main`). For linked list exercises (`t_list`), the subject will provide the struct definition; you do not submit it, but you must code against it.

Level 3 (Math, Algorithms, & Dynamic Allocation)

1. `add_prime_sum` (Program)

1. **Description:** Write a program that takes a positive integer as argument and displays the sum of all prime numbers inferior or equal to it followed by a newline. If the number of arguments is not 1, or if the argument is not a positive number, just display `0` followed by a newline.

2. **Prototype:** None (Program).

3. **Example Inputs/Outputs:**

- Input: `./add_prime_sum 5` | Output: `10\n` ($2 + 3 + 5 = 10$)
- Input: `./add_prime_sum 7` | Output: `17\n` ($2 + 3 + 5 + 7 = 17$)
- Input: `./add_prime_sum` | Output: `0\n`

2. `epur_str` (Program)

1. **Description:** Write a program that takes a string, and displays this string with exactly one space between words, with no spaces or tabs either at the beginning or the end, followed by a newline.

2. **Prototype:** None (Program).

3. **Example Inputs/Outputs:**

- Input: `./epur_str " See? It's easy to print the same thing "` |
Output: `See? It's easy to print the same thing\n`

- Input: `./epur_str " \t \t \t " | Output: \n`

3. `expand_str` (Program)

1. **Description:** Write a program that takes a string and displays it with exactly three spaces between each word, with no spaces or tabs at the beginning or the end, followed by a newline.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./expand_str " See? It's easy to print the same thing " |`
Output: `See? It's easy to print the same thing\n`

4. `ft_atoi_base` (Function)

1. **Description:** Write a function that converts the string argument `str` (base $N \leq 16$) to an integer (base 10) and returns it. The characters recognized in the input are: `0123456789abcdef` (and uppercase). You must handle an optional preceding minus sign.
2. **Prototype:** `int ft_atoi_base(const char *str, int str_base);`
3. **Example Inputs/Outputs:**

- Input: `"12F"`, base `16` | Returns: `303`
- Input: `"-1010"`, base `2` | Returns: `-10`

5. `ft_list_size` (Function)

1. **Description:** Write a function that returns the number of elements in the linked list that's passed to it.
2. **Prototype:** `int ft_list_size(t_list *begin_list);`
3. **Example Inputs/Outputs:**

- Input: A linked list with 5 nodes. | Returns: `5`
- Input: `NULL` | Returns: `0`

6. `ft_range` (Function)

1. **Description:** Write a function that allocates and returns an array of `int` s containing consecutive values beginning at `start` and ending at `end` (inclusive). If `start > end` , the array will go backwards.

2. **Prototype:** `int *ft_range(int start, int end);`

3. **Example Inputs/Outputs:**

- Input: `start = 1, end = 3` | Returns: `[1, 2, 3]`
- Input: `start = 0, end = -3` | Returns: `[0, -1, -2, -3]`

7. `ft_rrange` (Function)

1. **Description:** Write a function that allocates and returns an array of `int` s containing consecutive values starting at `end` and ending at `start` (inclusive).

2. **Prototype:** `int *ft_rrange(int start, int end);`

3. **Example Inputs/Outputs:**

- Input: `start = 1, end = 3` | Returns: `[3, 2, 1]`
- Input: `start = 0, end = -3` | Returns: `[-3, -2, -1, 0]`

8. `hiddenp` (Program)

1. **Description:** Write a program named `hiddenp` that takes two strings and displays `1` followed by a newline if the first string is hidden in the second one, otherwise displays `0` followed by a newline. Characters must be in the exact same order.

2. **Prototype:** None (Program).

3. **Example Inputs/Outputs:**

- Input: `./hiddenp "abc" "2altrb53c.sse"` | Output: `1\n`
- Input: `./hiddenp "abc" "btarc"` | Output: `0\n`

9. `lcm` (Function)

1. **Description:** Write a function that returns the lowest common multiple (LCM) of two unsigned integers. If either argument is 0, return 0.

2. **Prototype:** `unsigned int lcm(unsigned int a, unsigned int b);`

3. **Example Inputs/Outputs:**

◦ Input: `a = 15, b = 12` | Returns: `60`

10. `paramsum` (Program)

1. **Description:** Write a program that displays the number of arguments passed to it, followed by a newline.

2. **Prototype:** None (Program).

3. **Example Inputs/Outputs:**

◦ Input: `./paramsum 1 2 3 4` | Output: `4\n`

◦ Input: `./paramsum` | Output: `0\n`

11. `pgcd` (Program)

1. **Description:** Write a program that takes two positive string integers and displays their highest common denominator (HCF/GCD), followed by a newline.

2. **Prototype:** None (Program).

3. **Example Inputs/Outputs:**

◦ Input: `./pgcd 14 77` | Output: `7\n`

◦ Input: `./pgcd 17 3` | Output: `1\n`

12. `print_hex` (Program)

1. **Description:** Write a program that takes a positive (or zero) integer as a string, and prints it in base 16 (lowercase), followed by a newline.

2. **Prototype:** None (Program).

3. **Example Inputs/Outputs:**

◦ Input: `./print_hex 255` | Output: `ff\n`

◦ Input: `./print_hex 10` | Output: `a\n`

13. `rstr_capitalizer` (Program)

1. **Description:** Write a program that capitalizes the last letter of each word in a string and converts all other letters to lowercase. Handle multiple arguments by processing each on a new line.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./rstr_capitalizer "SecREt MessaGE"` | Output: `secreT messagE\n`

14. `str_capitalizer` (Program)

1. **Description:** Write a program that capitalizes the first letter of each word in a string and converts all other letters to lowercase. Handle multiple arguments by processing each on a new line.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./str_capitalizer "SecREt MessaGE"` | Output: `Secret Message\n`

15. `tab_mult` (Program)

1. **Description:** Write a program that displays a number's multiplication table from 1 to 9. Handle numbers passed as strings.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./tab_mult 9`

- Output:

```
1 x 9 = 9
2 x 9 = 18
...
9 x 9 = 81
```

Level 4 (Advanced Logic, Data Structures, & Linked Lists)

16. `fprime` (Program)

1. **Description:** Write a program that takes a positive integer and displays its prime factors on the standard output, followed by a newline. Factors must be displayed in ascending order and separated by `*`. If the number is 1, print `1`.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./fprime 225225` | Output: `3*3*5*5*7*11*13\n`
- Input: `./fprime 8333325` | Output: `3*3*5*5*11*11*31*97\n`

17. `ft_itoa` (Function)

1. **Description:** Write a function that takes an integer and returns a fresh string (allocated with `malloc`) representing the integer. Must handle negative numbers and `INT_MIN`.
2. **Prototype:** `char *ft_itoa(int nbr);`
3. **Example Inputs/Outputs:**

- Input: `-2147483648` | Returns: `"-2147483648"`
- Input: `0` | Returns: `"0"`

18. `ft_list_foreach` (Function)

1. **Description:** Write a function that takes a linked list and a function pointer. It must apply the given function to the data belonging to each node of the list.
2. **Prototype:** `void ft_list_foreach(t_list *begin_list, void (*f)(void *));`
3. **Example Inputs/Outputs:**

- Input: A list of string pointers, and a pointer to a print function. | Action: Prints every string in the list.

19. `ft_list_remove_if` (Function)

1. **Description:** Write a function that removes all nodes from a linked list whose data, when compared with `data_ref` using the function `cmp`, makes `cmp` return 0. You must free the memory of the removed nodes.
2. **Prototype:** `void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)());`
3. **Example Inputs/Outputs:** Not applicable (modifies linked list in place).

20. `ft_split` (Function)

1. **Description:** Write a function that splits a string of words into a null-terminated array of strings. Words are delimited by spaces, tabs, or newlines.
2. **Prototype:** `char **ft_split(char *str);`
3. **Example Inputs/Outputs:**

- Input: `"Hello world\\ttest"` | Returns: `["Hello", "world", "test", NULL]`

21. `rev_wstr` (Program)

1. **Description:** Write a program that takes a string and reverses the order of the words, separating them with a single space.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./rev_wstr "You hate people! But I love gatherings"`
- Output: `gatherings love I But people! hate You\\n`

22. `rostring` (Program)

1. **Description:** Write a program that takes a string and displays it with its first word moved to the end. Words should be separated by exactly one space, with no extra spaces at the beginning or end.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./rostring " hello world test " | Output: world test hello\n`
- Input: `./rostring "abc " | Output: abc\n`

23. `sort_int_tab` (Function)

1. **Description:** Write a function that sorts an array of integers in ascending order (in place).
2. **Prototype:** `void sort_int_tab(int *tab, unsigned int size);`
3. **Example Inputs/Outputs:**

- Input: `[5, 2, 8, 1, 3]` | Resulting Array: `[1, 2, 3, 5, 8]`

24. `sort_list` (Function)

1. **Description:** Write a function that sorts a linked list in ascending order based on the result of the `cmp` function pointer (returns 0 if ordered, non-zero if not ordered). The data pointers inside the elements should be swapped, not the elements themselves.
2. **Prototype:** `t_list *sort_list(t_list *lst, int (*cmp)(int, int));`
3. **Example Inputs/Outputs:** Not applicable (modifies linked list data in place).